Lab 3 report

实验目的与内容

搭建一个实现了部分指令的单周期 CPU

任务 1. 译码器设计

译码器设计就是狠狠地分类讨论。

```
localparam LU12I_W      =       7'b0001010;
localparam PCADDU12I    =       7'b0001110;
localparam SLTI         =       10'b0000001000;
localparam SLTUI        =       10'b0000001001;
localparam ADDI_W       =       10'b0000001010;
localparam ANDI         =       10'b0000001101;
localparam ORI          =       10'b0000001110;
localparam XORI         =       10'b0000001111;
localparam ADD_W        =       17'b00000000000100000;
localparam SUB_W        =       17'b00000000000100010;
localparam SLT          =       17'b00000000000100100;
localparam SLTU         =       17'b00000000000100101;
localparam AND          =       17'b00000000000101001;
localparam OR           =       17'b00000000000101010;
localparam XOR          =       17'b00000000000101011;
localparam SLL_W        =       17'b00000000000101110;
localparam SRL_W        =       17'b00000000000101111;
localparam SRA_W        =       17'b00000000000110000;
localparam SLLI_W       =       17'b00000000010000001;
localparam SRLI_W       =       17'b00000000010001001;
localparam SRAI_W       =       17'b00000000010010001;
localparam sel_rd0      =       1'b0;
localparam sel_pc       =       1'b1;
localparam sel_rd1      =       1'b0;
localparam sel_imm      =       1'b1;
```

首先定义局部参数。

```
assign    rf_ra0 = inst[31:25] == LU12I_W ? 5'b00000 : inst [ 9 : 5];
assign    rf_ra1 = inst [14 : 10];
assign    rf_wa  = inst [ 4 : 0];
```

本次实验中，rf_ra0、rf_ra1、rf_wa 可直接从 inst 中截取。

```
case(inst[31:25])
    LU12I_W:begin
        alu_op = `ALU_ADD;
        imm = {inst[24 : 5], {12{1'b0}}};
        rf_we = 1'b1;
        alu_src0_sel = sel_rd0;
        alu_src1_sel = sel_imm;
    end
    PCADDU12I:begin
        alu_op = `ALU_ADD;
        imm = {inst[24 : 5], {12{1'b0}}};
        rf_we = 1'b1;
        alu_src0_sel = sel_pc;
        alu_src1_sel = sel_imm;
    end
```

首先比对前 7 位，判断指令是否是 LU12I_W 或 PCADDU12I。若不是则比较前 10 位。

```
default:begin
    case(inst[31:22])
        SLTI:begin
            alu_op = `ALU_SLT;
            imm = {{20{inst[21]}}, inst[21:10]};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_imm;
        end
        SLTUI:begin
            alu_op = `ALU_SLTU;
            imm = {{20{inst[21]}}, inst[21:10]};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_imm;
        end
        ADDI_W:begin
            alu_op = `ALU_ADD;
            imm = {{20{inst[21]}}, inst[21:10]};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_imm;
        end
```

比较前 10 位，判断指令是否是立即数指令。这里截取部分片段以示说明。若不是立即数指令，比较前 17 位。

```
default:begin
    case(inst[31:15])
        ADD_W:begin
            alu_op = `ALU_ADD;
            imm = {32{1'b0}};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_rd1;
        end
        SUB_W:begin
            alu_op = `ALU_SUB;
            imm = {32{1'b0}};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_rd1;
        end
        SLT:begin
            alu_op = `ALU_SLT;
            imm = {32{1'b0}};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_rd1;
        end
        SLTU:begin
            alu_op = `ALU_SLTU;
            imm = {32{1'b0}};
            rf_we = 1'b1;
            alu_src0_sel = sel_rd0;
            alu_src1_sel = sel_rd1;
        end
```

```
SRAI_W:begin
    alu_op = `ALU_SRA;
    imm = {{27{1'b0}}, inst[14:10]};
    rf_we = 1'b1;
    alu_src0_sel = sel_rd0;
    alu_src1_sel = sel_imm;
end
default:begin//attention
    alu_op = 0;
    imm = 0;
    rf_we = 1'b0;
    alu_src0_sel = sel_rd0;
    alu_src1_sel = sel_imm;
end
```

若前 17 位也比对失败，则为未定义/未实现指令。全部输出信号置为默认值 0。

任务 2. 搭建 CPU

PC 模块：

```
always @(posedge clk) begin
    if(rst)begin
        pc <= 32'h1C000000;
    end else begin
        if(en)
            pc <= npc;
    end
end
```

PC_ADD4 模块：

```
module ADD4(
    input                   [31 : 0]        pc,
    output                  [31 : 0]        npc
);
assign npc = pc + 4;
endmodule
```

寄存器堆模块：

```
    reg [31 : 0] reg_file [0 : 31];
    // 用于初始化寄存器
    integer i;
    initial begin
        for (i = 0; i < 32; i = i + 1)
            reg_file[i] = 0;
    end

    assign rf_rd0 = reg_file [rf_ra0];
    assign rf_rd1 = reg_file [rf_ra1];
    assign dbg_reg_rd = reg_file [dbg_reg_ra];

    always @(posedge clk) begin
        if(rf_we) begin
            if(rf_wa != 5'b00000)
                reg_file[rf_wa] <= rf_wd;
        end
    end
```

最后在 CPU.v 中按照 datapath 连起来就可以了。

仿真：

```
stone_heart0027@DereksPC:~/COS_lab/Lab3/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time:                           1068
Instruction count:              533
PC:                             0x1C000850
Instruction:                    0x80000000
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x00000000 [ 2] = 0xB8231844 [ 3] = 0x5A973000
[ 4] = 0x00000DC1 [ 5] = 0x0000077D [ 6] = 0x9D692E90 [ 7] = 0x30568814
[ 8] = 0x0000077D [ 9] = 0x00000000 [10] = 0x95C44000 [11] = 0x8BD9C838
[12] = 0x73F62808 [13] = 0x8BD9CEB8 [14] = 0x00000840 [15] = 0xFFFFF6BC
[16] = 0x00000000 [17] = 0x4B61F7D8 [18] = 0x9D692820 [19] = 0x7274AEA4
[20] = 0x00000001 [21] = 0x30568E7C [22] = 0xB8231ECC [23] = 0x5A973000
[24] = 0x53BB6E2C [25] = 0xC3AD6E54 [26] = 0xFCDDF000 [27] = 0x7274A82C
[28] = 0x3FDB6E40 [29] = 0x0000086F [30] = 0x000003FE [31] = 0x00000001
```

上板：

```
User: R[illegible]AU----------

User: RR 0 32;
00000000 C036D7CC 0000082F 00000000
00000407 A81FB808 C036D4A4 00000000
A9D9B000 00000001 00000001 00000000
BFB4A4BC 00000000 00000000 35176000
00000000 BFB4A7F4 6E659000 048DA95B
00000643 BB0D3498 718544B0 5C703000
BB0D37B8 A81FB4C8 00000001 5C703000
A6503C01 917E1000 00000000 A9D9B000
```

R20 / T8 0×643	R21 0×bb0d3498	R22 / S9 / FP 0×718544b0	R23 / S0 0×5c703000
R24 / S1 0×bb0d37b8	R25 / S2 0×a81fb4c8	R26 / S3 0×1	R27 / S4 0×5c703000
R28 / S5 0×a6503c01	R29 / S6 0×917e1000	R30 / S7 0×0	R31 / S8 0×a9d9b000

如图，与 LARS 结果一致。

任务 3. 思考与总结

1. PC, REG_FILE

2. alu_src0 选择 pc: pcaddu12i

   alu_src0 选择 rf_rd0:addi.w

   alu_src1 选择 rf_rd1:add.w

   alu_src1 选择 imm:addi.w

3. 关键路径：cur_pc → imem_rdata → DECODE → REG_FILE → MUX1 → ALU → rf_wd。如果该路径的延迟大于一个周期，可能会导致寄存器堆回写错误、出现错误指令结果等结果。

总结

这次实验初步搭建了一个单周期 CPU，受益匪浅。